

ALPS: Measuring Invention in Language Models with Verified Mathematical Construction

Anonymous submission

Abstract

As large language models produce results presented as discoveries — new proofs, conjectures, molecules - the central question for AI-for-science is whether these outputs are invented or merely retrieved. No current evaluation can answer it: expert judgment is subjective, fixed problem sets contaminate training data, and tasks solvable by a general procedure reward compute rather than creativity. We present ALPS (Austin-Law Proof-Synthesis), a benchmark that removes all three confounds. Each instance is an equational law, machine-certified to have no nontrivial finite model, posing a two-sided task with a determinate answer: construct an infinite model satisfying the law, or prove that only the trivial model exists. A fresh solution must be invented for each law, since no finite search or uniform procedure reaches an infinite structure; answers are certified end to end with no human in the loop; and a public generator mints fresh instances at unbounded order, so no model has seen the problems it is scored on. Validating the design, a portfolio of the strongest automated provers resolves only 10% of the corpus, with a solve rate flat across an order of magnitude of compute — the surviving tier is bound by method, not budget — and our baseline settles an order-five case left open by the Equational Theories Project. Language models trace the same boundary: a frontier reasoning model achieves the first verified solves on the benchmark’s easiest deductive instances - solves that vanish as reasoning effort or support is reduced - while on the construction side, no model produces a single certified structure. A verified solve on ALPS is direct, machine-checked evidence of the capability existing benchmarks cannot measure: inventing a novel structure and proving it correct.

1 Introduction

As large language models (LLMs) are applied across mathematics and the sciences, AI-for-science has grown increasingly interested in evaluating LLM reasoning and creativity: whether what these models produce is discovered or merely retrieved. Recently, automated mathematical reasoning has become one of the most visible frontiers within AI-for-science, with systems now reaching gold-medal performance at the International Mathematical Olympiad (Google

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

DeepMind 2025). Two major recent developments sharpen this question, shifting it from solving problems towards producing discoveries. In 2026, a general-purpose reasoning model resolved the roughly eighty-year-old Erdős unit-distance problem, a long-open question in discrete geometry (OpenAI 2026). The SAIR Foundation’s Mathematics Distillation Challenge (Tao 2026), organized by Terence Tao and Damek Davis, asks competitors to design LLM systems that can construct proofs over the algebraic questions of the Equational Theories Project (Bolan et al. 2025). The common thread is a standard that the rest of AI-for-science rarely meets: a creative answer only counts once it has been proven correct.

Proven correct is not the same as new. As LLMs begin to take on scientific work - formalizing and proving theorems (Yang et al. 2023; Xin et al. 2024), proposing conjectures (Romera-Paredes et al. 2024; Gottweis et al. 2026; Lu et al. 2024), and designing molecules and materials (Jumper et al. 2021; Merchant et al. 2023) - the essential question becomes whether the LLM’s output is invented or reproduced. No current evaluation can tell these apart. Judging LLM outputs by human experts or physical experiment is slow, costly, and subjective. Scoring against reference answers is objective, but answers unavoidably contaminate LLM training data (Xu et al. 2024; Chen et al. 2025), so a high score may reflect recall rather than discovery. When a known general procedure solves the task, success cannot be attributed to the model at all: executing a procedure is a matter of compute, not creativity. What is missing is a benchmark that isolates invention - novel work scored objectively, on problems no model has seen before and no known procedure reaches.

Formal mathematics is the one setting where such a benchmark can be engineered: correctness can be certified by a machine rather than judged by a human, and fresh problems can be minted rather than curated. We build ours within equational theory (Birkhoff 1935), the study of *magmas*: sets with a single binary operation $\diamond$ (Burris, Sankappanavar et al. 1981). Its *laws* are universally quantified equations that an operation may or may not obey, such as commutativity $x \diamond y = y \diamond x$ (Burris, Sankappanavar et al. 1981; Baader and Nipkow 1998). A magma is a *model* of a law when the equation holds under every assignment of its elements (Burris, Sankappanavar et al. 1981). The one-element magma models every law and thus carries no information; it is named

the *trivial* magma (Burris, Sankappanavar et al. 1981). What carries content is whether a law has a nontrivial model, one with at least two elements. Within a finite set, such a model can always be found by search, since only finitely many operations exist to test. However, some laws have no nontrivial finite model at all. Their only nontrivial models are infinite, and no search through finite candidates can reach an infinite structure (Kisielewicz 1997; Bolan et al. 2025). For these laws, finding a model stops being a matter of computation and becomes a matter of invention: an infinite structure cannot be enumerated, only conceived, then justified.

We build the benchmark from the laws that require this invention: those with no nontrivial finite model, a property that we verify in advance. With finite models excluded, each law falls into one of two cases: either it forces every model to be trivial, entailing $x = y$, or it has a nontrivial model, which must then be infinite. Laws of the second kind, an infinite model but no finite one, are the Austin laws (Kisielewicz 1997).

Austin laws sit where AI-for-science and mathematical provers are converging. The recent Erdős unit-distance result was settled by a reasoning model through the construction of an infinite object, the kind of scientific leap that marks the frontier of what today’s language models can do (OpenAI 2026). Computational mathematical provers approach this same frontier from the other side, with recent progress demonstrating new methods of constructing infinite models (Janota, Rawson, and Schulz 2026). Austin laws are this challenge in renewable form: each is an infinite-construction problem with a definite answer, and fresh ones can be generated without limit. It is that combination, a frontier problem that is nonetheless checkable and endless in supply, that makes them something a benchmark can be built upon.

We call this benchmark Austin-Law Proof-Synthesis (ALPS). Each problem is one such law, and solving one means providing a proof showing which of the two cases holds true. A solver must either establish that the law is Austin, by constructing an infinite model and showing it satisfies the law, or prove that the law entails $x = y$, meaning that all models for this law must be trivial.

What ALPS requires of the solver is a capability that reaches well beyond equational theory. Resolving a law requires proposing a completely novel structure, then arguing from the law alone that it satisfies a specification over infinitely many cases it can never check directly. The steps taken to solve an ALPS instance are the exact same as those in scientific discovery (Figure 1). First comes specification: the constraints an acceptable answer must satisfy, such as a target profile for a molecule, or a single law for ALPS. Then comes invention: reasoning over how those constraints interact to construct a candidate that meets all of them at once. Last comes validation, an independent check that settles whether the candidate satisfies its specification. In ALPS, that validation is a formal proof, so the creative leap is isolated and made objectively measurable. We use “invention” in the operational sense. ALPS judges whether a model can produce an answer that is novel, verifiably correct, and derivable by no known procedure, not whether anything resembling human creativity occurred inside the model, judged on the answer,

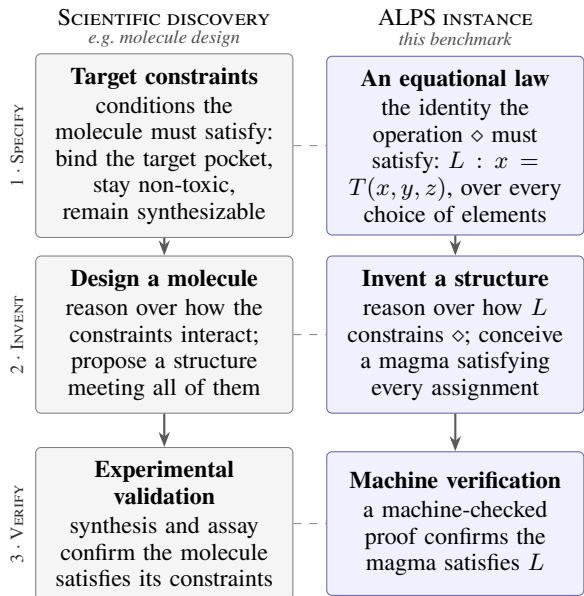


Figure 1: The three phases of inventive work - specify, invent, verify - in scientific discovery (left, instantiated here by molecule design) and an ALPS instance (right). In both, a specification defines the requirements of an acceptable answer, a candidate must be invented to satisfy all of them at once, and a final check settles whether it does. Because laws can be generated without limit, every ALPS instance can be one no model has seen.

not the process. For example, a solver might notice that a fresh law resembles a known Austin law and adapt a known construction to fit it. A successful adaptation is itself invention by our standard. It requires deciding which construction to reach for and how to reshape it until every constraint of the new law holds. In this case, the solver has produced a tailored solution rather than executed a known procedure - one that applies identically to every problem, leaving the LLM with nothing to decide itself. These problems are exceedingly difficult, but that is the intent: progress on them is direct evidence of the ability to construct and verify genuinely new structure - the capability that separates discovery from recall, and the one current benchmarks cannot measure.

Three properties make this measurement clean, one for each confound, shown in Table 1. ALPS is verifiable: each answer is settled by a certificate a machine can check, so grading is human-free and provably correct. It is renewable: fresh instances can be produced without limit, so evaluation always has problems no model has seen. And it demands creative construction: a nontrivial model exists only among infinite structures and cannot be reached with a general procedure, so solutions must be invented for each law. Every prior benchmark family has at most two of the three, and the missing property results in a confounded evaluation of invention. Synthetic puzzles such as random SAT (Selman, Mitchell, and Levesque 1996) and the Equational Theories Project (Bolan et al. 2025) are verifiable and renewable, but their problems

	Verifiable <i>no subjectivity</i>	Renewable <i>no contamination</i>	Constructive <i>no set procedure</i>
Static problem sets MATH, code, miniF2F	✓		✓
Synthetic puzzles random SAT, ETP	✓	✓	
Open discovery hypotheses, molecules		✓	✓
ALPS (ours)	✓	✓	✓

Table 1: Each property removes one confound in measuring invention: verification removes subjective judging, renewal removes contamination, and construction removes solution by a general procedure. Each prior benchmark family (Hendrycks et al. 2021; Chen et al. 2021; Zheng, Han, and Polu 2021; Selman, Mitchell, and Levesque 1996; Bolan et al. 2025; Gottweis et al. 2026; Jumper et al. 2021) fulfills two of the three, but measuring invention requires all three.

are solved by a general procedure given enough compute, so success instead measures an LLM’s ability to execute a known task. Static corpora such as MATH (Hendrycks et al. 2021), coding benchmarks (Chen et al. 2021), and miniF2F (Zheng, Han, and Polu 2021) ask for constructed, checkable solutions, but come from fixed sets that eventually contaminate, so a high score may reflect memorization. Open scientific discovery, such as hypothesis generation (Gottweis et al. 2026; Lu et al. 2024) or molecule design (Jumper et al. 2021), demands construction and never exhausts its supply, but its answers are judged subjectively, so scores are dependent on the judge. ALPS removes each of these confounds. A solver scored on ALPS did not find the answer through a known procedure, did not recall a seen answer, and did not persuade a judge: it invented its answer and proved it correct.

To validate that no known procedure reaches these problems, we construct a portfolio of the strongest automated provers (Kovács and Voronkov 2013; Schulz, Cruanes, and Vukmirović 2019; Smallbone 2021), finding that it resolves only a small part of ALPS. A large and growing set of problems resists it entirely, yielding no model at any compute budget we test, even though every instance is certified to have a determinate answer. The solve rate does not increase with compute: this difficulty is bound by method, not computational budget. In full, our contributions are:

- a two-sided construction task on Austin laws, with every instance certified to have a determinate answer;
- a corpus of such laws and a public generator that mints fresh instances without limit;
- an automated judge that certifies submissions through independent channels - a Lean proof (Moura and Ullrich 2021) for triviality, and an automated-prover certificate (Kovács and Voronkov 2013) for constructed models - with no human in the loop; and
- a strong automated baseline that establishes the method-bound hard tier and, along the way, settles an order-5 case left open by the Equational Theories Project (ETP)

(Bolan et al. 2025).

2 Related Works

2.1 AI-for-Science Benchmarks

Existing benchmark families each hold two of the three properties ALPS targets and fall short on the third. Static problem sets are fixed collections spanning grade-school and competition mathematics (Cobbe et al. 2021; Hendrycks et al. 2021), graduate science (Rein et al. 2023), coding (Chen et al. 2021), and formal proof construction (Zheng, Han, and Polu 2021; Azerbayev et al. 2023; Tsoukalas et al. 2024). These are scored objectively and require reasoning through each problem rather than recalling a memorized algorithm. However, because their problems are fixed, these benchmarks inevitably suffer from contamination, as their evaluation set will eventually enter the training data of the language models they are meant to test, rendering further evaluation on this benchmark unreliable (Xu et al. 2024; Chen et al. 2025). Some benchmarks attempt to delay contamination by holding problems secret (Glazer et al. 2024) or releasing new problems over time (White et al. 2025; Jain et al. 2025), but will always be limited by the rate at which humans can author new problems.

Synthetic puzzles avoid this limit by generating problems from a fixed pattern or rule so that fresh instances can be produced without end, such as random SAT formulas (Selman, Mitchell, and Levesque 1996), graph-generated reasoning tasks (Zhu et al. 2024), Sudoku puzzles (Seely et al. 2025), or the equational laws of the Equational Theories Project (Bolan et al. 2025). However, the same regularity that makes these problems generable also makes them solvable by a known procedure. Therefore, these benchmarks often test the LLM’s ability to execute a known method rather than to truly construct a creative solution.

Open scientific discovery benchmarks evaluate an LLM’s ability to construct novel, useful scientific advancements such as hypotheses, conjectures, designs, and mathematical structures (Lu et al. 2024; Gottweis et al. 2026; Alkan et al. 2025; Romera-Paredes et al. 2024; Merchant et al. 2023; Jumper et al. 2021). These pose a challenging task, requiring the LLM to invent a new solution rather than retrieve or execute a known procedure. However, their results are validated by human expertise or physical experiment, making evaluation subjective and slow. ALPS uniquely combines the strengths of each benchmark family: objective scoring, endless renewal, and a demand for genuine construction.

2.2 Theorem Proving and Verification

ALPS scores answers using machine verification, without a human in the loop, made possible by theorem provers. Lean (Moura and Ullrich 2021) is a functional programming language equipped with a proof assistant whose kernel can automatically check the validity of proofs, such that a certificate is guaranteed to be correct. Each proposition, including the statement that a magma satisfies a law, is encoded as a type within the language, and a proof of it is a term inhabiting that type, exactly as a value inhabits a type in an ordinary programming language. Checking a proof is resolved using

type checking: the kernel confirms that submitted terms have the claimed type, similar to a compiler checking that a function’s body matches its signature. A statement quantified over infinitely many inputs, such as a law holding under every assignment, becomes a function type. Within Lean, its proof is a function returning a proof for an arbitrary argument, and the kernel type-checks that function once, symbolically. A certificate that type-checks successfully is therefore correct by construction and automatically validated.

Unlike checking them, finding proofs is difficult. Many works investigate automating this search. Classical automated theorem provers such as Vampire and E (Kovács and Voronkov 2013; Schulz, Cruanes, and Vukmirović 2019) search for first-order proofs by superposition calculus (Bachmair and Ganzinger 1994), a strategy that repeatedly derives logical consequences of the input until it reaches a contradiction or no new consequences remain. More recently, language models have been trained to construct proofs in systems like Lean (Yang et al. 2023; Xin et al. 2024; Hubert et al. 2026), measured on formal proof benchmarks (Zheng, Han, and Polu 2021).

2.3 Equational Reasoning

Equational theory studies the foundation of algebra - the identities that operations satisfy and the entailments among them. The Equational Theories Project (Bolan et al. 2025) studies this at scale, enumerating the implications among thousands of small laws and settling them automatically using a pipeline of automated provers and finite-model finders. This work singles out the laws whose only nontrivial models are infinite, the Austin laws (Kisielewicz 1997), as the hardest to resolve, since neither a searchable finite model nor a short derivation settles them.

Several automated methods are tailored to equational laws. Paradox and Mace4 (Claessen and Sörensson 2003; McCune 2003) are designed to find finite models when they exist, while Infinox (Claessen and Lillieström 2011) instead proves that no finite model exists. Twee (Smallbone 2021) implements unfailing Knuth-Bendix completion (Knuth and Bendix 1970; Bachmair, Dershowitz, and Plaisted 1989), orienting equations into rewrite systems to solve equational problems. When a saturation from Twee or a superposition prover halts without reaching a contradiction, the clauses it has accumulated already describe a model. Janota, Rawson, and Schulz (2026) recently studied this for equational theories, extracting an explicit model from the saturation in a case study.

3 The Austin-Law Proof-Synthesis Benchmark

ALPS is built in three parts, displayed in Figures 2 and 3 : a two-sided task posed over laws certified to have no nontrivial finite model, a corpus construction that mints such laws at any order by extending known Austin laws and screening the results, and an automated judge that verifies answers to either side of the task with a machine-checked certificate. We describe each component below. Each part describes one column of Table 1: the admissibility certificate makes every

instance a construction problem with a determinate answer, the extension engine renews the supply without limit, and the judge verifies answers with no human in the loop.

3.1 Problem Formulation

A magma is a set M , called the carrier, together with a single binary operation $\diamond: M \times M \rightarrow M$ and no further axioms. An equational law is a universally quantified identity between two terms built from variables and $\diamond$. Its order is the number of times the operation appears, so the commutative law $x \diamond y = y \diamond x$ has order two. A magma *satisfies* a law when the identity holds under every assignment of carrier elements to its variables, and is then called a *model* of the law. Every ALPS instance is a law of the form $L: x = T[x, y, z, \dots]$, a single variable equated with a term T of order five or more. The trivial magma contains only one element, and is thus a model of every law, carrying no information. We call a model nontrivial when its carrier holds at least two distinct elements. The question ALPS poses for each L is whether a nontrivial model exists at all: either L entails $x = y$, collapsing every model to a single value, or some nontrivial magma satisfies it.

A nontrivial model, when one exists, may be finite or infinite. The finite case is a matter of search: enumerating operations on carriers of increasing size must eventually find it, requiring none of the law-specific creativity the benchmark aims to measure. We therefore admit a law into the benchmark only when it carries a machine-checked proof that no nontrivial finite model exists, calling such laws *admissible*. Admissibility forces a dichotomy - every law either entails $x = y$ or it does not, so each instance has a determinate answer. For an admissible law, the two cases are exactly

$$\underbrace{L \models x = y}_{\text{trivial}} \quad \text{or} \quad \underbrace{L \text{ has an infinite nontrivial model}}_{\text{Austin}} \quad (1)$$

Laws of the second kind are the Austin laws (Kisielewicz 1997).

The asymmetry between the two sides of the task is essential to what the benchmark measures. Whether L entails $x = y$ is a first-order consequence, so a complete proof search will confirm it whenever it holds. The trivial side is, in principle, settled by deduction. The Austin case admits no such guarantee. A search that fails to derive $x = y$, however long it runs, is not a model. Because equational entailment is undecidable in general (Baader and Nipkow 1998), no uniform procedure can be expected to close the gap. Establishing that a law is Austin means inventing an infinite magma tailored to the individual law to satisfy it. This synthesis, which deductive search does not supply, is the capability that ALPS isolates.

3.2 Corpus Construction

Generation by extension. Admissible laws are too rare for random sampling to produce in quantity, and the few it does are often redundant. We instead grow the corpus outward from laws already known to be Austin (Figure 2): a seed law $x = T$ is extended by selecting any variable v in T

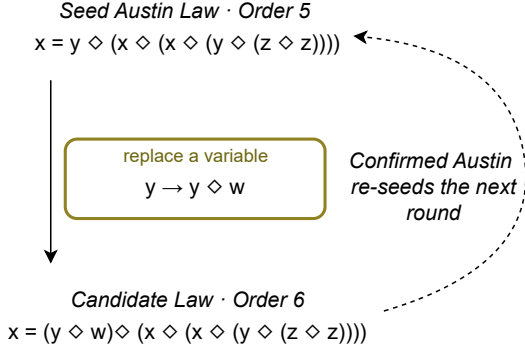


Figure 2: Seed Austin laws are extended by replacing one variable occurrence v with $v \diamond w$ or $w \diamond v$. Laws confirmed Austin re-seed the next round, so generation applies recursively at every order.

and replacing one occurrence with $v \diamond w$ or $w \diamond v$ for some other variable w , yielding higher-order candidates. Because a candidate inherits most of the seed’s term structure, it often inherits the mechanism that prevents a finite model, increasing the likelihood of finding new admissible laws relative to random sampling. The construction is recursive - laws confirmed Austin at order n seed the round at order $n + 1$ - so the corpus has no terminal order. This enables ALPS to be renewable: fresh instances can be minted after any training cutoff, so evaluation never depends on problems a model may have seen.

Screening. Each candidate then passes the screening pipeline shown in Figure 3. A finite-model filter searches carriers of increasing size, discarding every candidate it can satisfy nontrivially. The survivors face the admissibility prover, a first-order query whose success is a proof that no nontrivial finite model exists: when some subterm of T contains every occurrence of x , that subterm acts as a left inverse, making the map $x \mapsto T$ injective. On a finite carrier, an injective map is surjective, and asserting that surjectivity lets the prover derive $x = y$. Admission requires this proof, and no such proof exists for a law with a finite model. The cost is incompleteness: when injectivity cannot be proven within a candidate, nothing is certified. Such candidates are discarded, even though some may be admissible. The admissible pool that remains is split by (1). The screening provers classify the pool into laws proven trivial, laws proven Austin, and the residual that the baseline of the next section attacks. This certificate closes the search route: an admitted law cannot be settled by enumerating finite candidates, only by inventing a model or proving triviality.

Deduplication. Because every law descends from a small seed set, extensions risk equivalence with their seed or with a sibling. For the laws proved Austin, the models found during screening can prove distinctness at no prover cost. A magma in which one law holds and the other fails is already a certificate that the two are distinct. Of the 34,191 pairs across

the 262 laws proved Austin, 33,936 are separated by such a model, while 255 are proved equivalent by the prover, with none left undecided. Closing the 255 equivalent pairs under transitivity merges away 67 laws, so the 262 laws collapse to 195 classes. We report corpus size in classes rather than laws so the count only reflects distinct problems, so no invention is credited twice.

3.3 Answer Verification

A solved ALPS instance is a verified mathematical fact, not a predicted label. The two sides of the task call for different verification processes, so the judge issues a certificate through the appropriate channel: a proof checked by the Lean kernel for the trivial side, and an automated-prover certificate for the constructed model. Both channels are fully automatic, and a submission is accepted only if its check succeeds.

The trivial channel. For each law the judge generates the proposition

$$\text{TRIVIALGOAL} \equiv \forall M \forall \diamond : M \times M \rightarrow M, \quad \text{Law}(\diamond) \rightarrow \forall a, b : M, a = b, \quad (2)$$

where Law asserts that $\diamond$ satisfies L , and a submission is a Lean proof of it. The judge generates the statement, and the submitted proof must elaborate at exactly that type, so a proof the kernel accepts is correct by construction (Moura and Ullrich 2021).

The construction channel. A solver that believes L is Austin does not write a Lean proof about an infinite object. It submits the model itself, as a finite presentation E - a set of equations over $\diamond$ that pins down the intended structure. The judge then runs two independent prover queries (Kovács and Voronkov 2013):

$$(a) E \vdash L \quad (b) E \cup \{a \neq b\} \text{ is satisfiable}, \quad (3)$$

certifying the submission only when both succeed. Query (a) is a refutation proof that every model of E satisfies L ; query (b) is a terminating saturation establishing that E has a model with two distinct elements. Together they exhibit a nontrivial model of L , which admissibility forces to be infinite, so L is Austin.

A single Lean judge for both sides is not available, for a reason that strengthens the benchmark’s premise. If an admissible law had a model whose operation is a polynomial or affine formula over $\mathbb{Z}$, $\mathbb{Z}^k$, or $\mathbb{Z}/n$, the identity would survive reduction modulo two and yield a nontrivial finite model, contradicting admissibility. No Austin model is an arithmetic formula, so the model families that are straightforward to formalize in Lean are provably empty here. Term models built from saturations could validate such a proof in Lean, but they await a formalization of ground confluence (Janota, Rawson, and Schulz 2026).

4 Experiments

4.1 The Automated Frontier

We first establish what automation resolves. We construct a portfolio spanning eight configurations of the strongest off-the-shelf provers: Vampire 5.0.1 in five modes (proof search and saturation, under Knuth-Bendix and lexicographic path

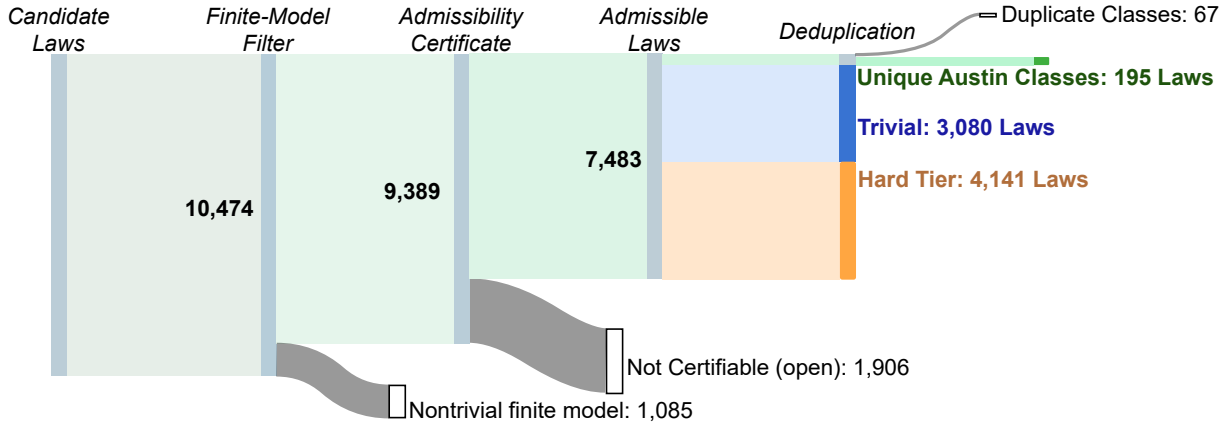


Figure 3: Each generated candidate passes several screens to guarantee quality. A finite-model filter discards laws with small nontrivial models. The admissibility prover certifies the rest have no finite model and discards any that it cannot certify, labeled “(open)” to signify that some discarded candidates may be admissible. Proofs generated throughout the screening process help classify the admissible law pool into trivial, Austin, or the unresolved hard tier.

orderings) (Kovács and Voronkov 2013), the E prover in a proof and a saturation mode (Schulz, Cruanes, and Vukmirović 2019), and Twee 2.6.1, an implementation of un-failing completion built for equational reasoning (Smallbone 2021), each running over per-prover budgets from 30 to 600 seconds. A law counts as resolved when any configuration returns a certificate, and a claimed model or triviality proof is one the tool itself reports. Further details of the portfolio are explained in the Appendix. Finite-model finders such as Mace4 (?) and Paradox (?) are omitted. Our admissibility classification certifies that no nontrivial model exists, so they cannot succeed. Saturation and completion are the automated methods able to attempt infinite-model construction at all.

Sweeping the portfolio over the admissible pool produces two populations with almost nothing between them: laws that resolve nearly instantly, and laws that do not resolve at all. Every one of the 262 Austin models the screening pass constructs arrives in a median of 0.1 seconds; the full sweep of the 4,141-law residual recovers only a thin further layer — [FILL: 44] late triviality proofs and [FILL: 4] completion-only models, about [FILL: 3]% of laws swept, reassigned to their proper tiers by the two-sided design — and nothing else, at any budget. The [FILL: final] laws that remain are the hard tier, and the boundary is not a threshold we chose: it is a gap in the distribution.

The floor is not without content. Among the laws the portfolio does resolve is an order-five law and its dual (12857 and 33436) that the Equational Theories Project recorded as having only trivial finite models while leaving the existence of an infinite model unresolved. Our pipeline confirms the pair Austin by two independent routes — a Vampire saturation and a Twee completion — and the extracted 27-rule model is exercised directly: every rule fires, and $x = y$ is refuted in it. The completed system is not orientable into a plain rewrite system, which is precisely the ground-confluence obstacle that keeps such models outside Lean’s reach today; the full

worked example is in the appendix. The pipeline, pointed at the literature, returns a new fact — but the portfolio reaches it, so the case marks the floor’s edge, not the frontier.

If difficulty were graded by compute, added budget would buy resolutions along that gradient. It does not (Table ??). On a 120-law sample re-attacked with all eight configurations, three laws resolve at the 30-second rung — all trivial — and none at any higher rung, out to roughly 170× the screening pass’s per-law time; a separate retry of 294 unresolved laws at ten times the budget converts 3.7%, all to trivial, none to a model. A law yields in fractions of a second or resists everything we run: the difficulty is bound by method, not budget.

Nor is the difficulty one prover’s blind spot. Superposition and completion explore the space differently enough that each resolves instances the other diverges on — Twee completes in seconds an order-five law whose saturation runs Vampire to hundreds of clauses, and the sweep’s recovered models are completion-only. Yet the two paradigms’ reach ends in the same place: past the shed contamination, neither constructs a model the other misses, and the residual resists both. The frontier is a property of the laws, not of a tool.

4.2 The LLM Frontier

Language models enter ALPS on the same terms as the portfolio: nothing in the task references how an answer is produced, only that it certifies. We evaluate them under a fixed protocol designed so that failures are attributable. Every model receives identical inputs — the law, the task statement, and a fixed answer format — and interacts with the judge as a verification oracle over three feedback rounds. On the trivial side the formalization burden is removed entirely: the model emits only a chain of terms, each step one application of the law; the harness matches each step, bridges short gaps, and assembles the Lean proof itself, so the test is finding the derivation, not writing Lean. Support is graded

Model	Certified-easy ($n = 63$)			Hard ($n = 25$)
	full support	no waypoints	low effort	construction
GPT-4.1	0	0*	—	—
o4-mini	0	0*	0*	—
o3	9	4*	0*	0

Table 2: Verified LLM solves under the fixed protocol (identical inputs, three feedback rounds against the judge). Full support = waypoint lemmas in the prompt, medium reasoning effort; trivial-side solves are Lean-verified, construction submissions ATP-certified. *Reduced-support settings rerun only the laws solved at full support; since waypoints and reasoning effort can only help, laws unsolved at the stronger setting are counted as unsolved. Dashes: setting not applicable or not run.

— runs range from the bare law to full reveal, where prover-derived waypoint lemmas are included in the prompt — and the harness is frozen across models. Saturation provers are withheld from the models, since with them a model would simply re-run the portfolio at larger budget; the judge and assembler are exactly the tools any solver built on ALPS would use.

Instance selection is the step we control most carefully. From the trivial tier we isolate a *certified-easy* set of 63 laws: those whose complete collapse chains a blind bidirectional search finds in under five seconds, most only three law applications long. Each carries a privately held, machine-found solution, and one such chain, driven through the assembler and the Lean kernel, serves as an end-to-end positive control. A zero on this set cannot be blamed on the instances — the answers exist, are short, and are found by uninformed search almost instantly. On the construction side we draw a 25-law sample of the hard tier; there the model proposes a presentation E and the ATP certificate of the verification section grades it.

Table 2 reports the outcomes by the question each block answers. On the certified-easy set at full support, o3 at medium reasoning effort produces nine kernel-verified solves — to our knowledge the first machine-checked language-model solutions to ALPS instances — while GPT-4.1 and o4-mini produce none on the same 63 laws under the identical frozen protocol. The nine solutions are chains of three or four law applications, most found on the second or third feedback round at a median of roughly 50,000 completion tokens per solved law: the verification oracle does real work, and the capability tracks reasoning tier rather than vendor or prompt.

Ablating o3 on the nine laws it solves locates what the solves depend on. With the waypoint lemmas withheld, four of the nine survive — the model finds some short derivations entirely unaided — and with reasoning effort reduced to low, none do. Solves exist only toward the top of both dials, and rejected chains fail overwhelmingly the same way at every setting: they propose steps that are not instances of the law at all, the failure that eliminates the weaker models outright.

The construction side stays at zero, and the way it stays there is the finding. In 23 of 25 attempts, the presentation o3 proposes entails the law — the correctness query passes — but fails nontriviality: the proposed structure collapses to a point, or its saturation does not terminate. The remaining two fail the converse way, consistent but too weak to force the law. Every attempt satisfies one horn of the certificate;

Resolutions by per-config budget	
30 s	105
60 s	4
120 s	1
300 s	1
600 s	0

Table 3: Number of new laws solved per portfolio compute budget. 105/4,141 laws are resolved at the first rung. Running the portfolio with $20\times$ the compute solves only six additional laws.

Portfolio sweep of the 4,141-law residual			
proved trivial	102	2.5	
nontrivial model found	9	0.2	
unresolved — the hard tier	4,030	97.3	

Table 4: Classification of solved laws after sweeping the solver portfolio over every law that survived screening.

none satisfies both. Inventing a structure strong enough to entail the law yet loose enough to keep two elements apart is precisely the synthesis the benchmark isolates, and nothing we ran — prover, completion, or reasoning model — achieves it. The asymmetry of the task thus reappears one level up: the deductive side yields, modestly and under support, to the strongest reasoning model we test; the construction side yields to nothing.

5 Discussion

References

- Alkan, A. K.; Sourav, S.; Jablonska, M.; Astarita, S.; Chakrabarty, R.; Garuda, N.; Khetarpal, P.; Pióro, M.; Tanoglidis, D.; Iyer, K. G.; et al. 2025. A survey on hypothesis generation for scientific discovery in the era of large language models. *arXiv preprint arXiv:2504.05496*.
- Azerbaiyev, Z.; Piotrowski, B.; Schoelkopf, H.; Ayers, E. W.; Radev, D.; and Avigad, J. 2023. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics. *arXiv preprint arXiv:2302.12433*.
- Baader, F.; and Nipkow, T. 1998. *Term rewriting and all that*. Cambridge university press.

- Bachmair, L.; Dershowitz, N.; and Plaisted, D. A. 1989. Completion without failure. In *Rewriting Techniques*, 1–30. Elsevier.
- Bachmair, L.; and Ganzinger, H. 1994. Rewrite-based equational theorem proving with selection and simplification. *Journal of Logic and Computation*, 4(3): 217–247.
- Birkhoff, G. 1935. On the structure of abstract algebras. In *Mathematical proceedings of the Cambridge philosophical society*, volume 31, 433–454. Cambridge University Press.
- Bolan, M.; Breitner, J.; Brox, J.; Carlini, N.; Carneiro, M.; van Doorn, F.; Dvorak, M.; Goens, A.; Hill, A.; Husum, H.; et al. 2025. The Equational Theories Project: Advancing collaborative mathematical research at scale. *arXiv preprint arXiv:2512.07087*.
- Burris, S.; Sankappanavar, H. P.; et al. 1981. *A course in universal algebra*, volume 78. Springer.
- Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H. P. D. O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Chen, S.; Chen, Y.; Li, Z.; Jiang, Y.; Wan, Z.; He, Y.; Ran, D.; Gu, T.; Li, H.; Xie, T.; et al. 2025. Recent advances in large language model benchmarks against data contamination: From static to dynamic evaluation. *arXiv preprint arXiv:2502.17521*.
- Claessen, K.; and Lillieström, A. 2011. Automated inference of finite unsatisfiability. *Journal of Automated Reasoning*, 47(2): 111–132.
- Claessen, K.; and Sörensson, N. 2003. New techniques that improve MACE-style finite model finding. In *Proceedings of the CADE-19 Workshop: Model Computation-Principles, Algorithms, Applications*, 11–27.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; et al. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Glazer, E.; Erdil, E.; Besiroglu, T.; Chicharro, D.; Chen, E.; Gunning, A.; Olsson, C. F.; Denain, J.-S.; Ho, A.; Santos, E. d. O.; et al. 2024. Frontiermath: A benchmark for evaluating advanced mathematical reasoning in ai. *arXiv preprint arXiv:2411.04872*.
- Google DeepMind. 2025. Advanced Version of Gemini with Deep Think Officially Achieves Gold-Medal Standard at the International Mathematical Olympiad. <https://deepmind.google/blog/advanced-version-of-gemini-with-deep-think-officially-achieves-gold-medal-standard-at-the-international-mathematical-olympiad/>.
- Gottweis, J.; Weng, W.-H.; Daryin, A.; Tu, T.; Sirkovic, P.; Myaskovsky, A.; Glowaty, G.; Weissenberger, F.; Orlandi, A.; Popovici, D.; et al. 2026. Accelerating scientific discovery with Co-Scientist. *Nature*, 1–3.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.
- Hubert, T.; Mehta, R.; Sartran, L.; Horváth, M. Z.; Žužić, G.; Wieser, E.; Huang, A.; Schrittwieser, J.; Schroecker, Y.; Ma-soom, H.; et al. 2026. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651(8106): 607–613.
- Jain, N.; Gu, A.; Li, W.-D.; Yan, F.; Zhang, T.; Wang, S.; Solar-Lezama, A.; Sen, K.; and Stoica, I. 2025. Live-codebench: Holistic and contamination free evaluation of large language models for code. In *International Conference on Learning Representations*, volume 2025, 58791–58831.
- Janota, M.; Rawson, M.; and Schulz, S. 2026. Case Study: Saturations as Explicit Models in Equational Theories. *arXiv preprint arXiv:2602.16324*.
- Jumper, J.; Evans, R.; Pritzel, A.; Green, T.; Figurnov, M.; Ronneberger, O.; Tunyasuvunakool, K.; Bates, R.; Žídek, A.; Potapenko, A.; et al. 2021. Highly accurate protein structure prediction with AlphaFold. *nature*, 596(7873): 583–589.
- Kisielewicz, A. 1997. Austin identities. *algebra universalis*, 38(3): 324–328.
- Knuth, D. E.; and Bendix, P. B. 1970. Simple word problems in universal algebras. In *Computational problems in abstract algebra*, 263–297. Elsevier.
- Kovács, L.; and Voronkov, A. 2013. First-order theorem proving and Vampire. In *International Conference on Computer Aided Verification*, 1–35. Springer.
- Lu, C.; Lu, C.; Lange, R. T.; Foerster, J.; Clune, J.; and Ha, D. 2024. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.
- McCune, W. 2003. Mace4 reference manual and guide. *arXiv preprint cs/0310055*.
- Merchant, A.; Batzner, S.; Schoenholz, S. S.; Aykol, M.; Cheon, G.; and Cubuk, E. D. 2023. Scaling deep learning for materials discovery. *Nature*, 624(7990): 80–85.
- Moura, L. d.; and Ullrich, S. 2021. The lean 4 theorem prover and programming language. In *International Conference on Automated Deduction*, 625–635. Springer.
- OpenAI. 2026. An OpenAI Model Has Disproved a Central Conjecture in Discrete Geometry. <https://openai.com/index/model-disproves-discrete-geometry-conjecture/>.
- Rein, D.; Hou, B. L.; Stickland, A. C.; Petty, J.; Pang, R. Y.; Dirani, J.; Michael, J.; and Bowman, S. R. 2023. Gpqa: A graduate-level google-proof q&a benchmark. *arXiv preprint arXiv:2311.12022*.
- Romera-Paredes, B.; Barekatin, M.; Novikov, A.; Balog, M.; Kumar, M. P.; Dupont, E.; Ruiz, F. J.; Ellenberg, J. S.; Wang, P.; Fawzi, O.; et al. 2024. Mathematical discoveries from program search with large language models. *Nature*, 625(7995): 468–475.
- Schulz, S.; Cruanes, S.; and Vukmirović, P. 2019. Faster, higher, stronger: E 2.3. In *International Conference on Automated Deduction*, 495–507. Springer.
- Seely, J.; Imajuku, Y.; Zhao, T.; Cetin, E.; and Jones, L. 2025. Sudoku-Bench: Evaluating creative reasoning with Sudoku variants. *arXiv preprint arXiv:2505.16135*.

- Selman, B.; Mitchell, D. G.; and Levesque, H. J. 1996. Generating hard satisfiability problems. *Artificial intelligence*, 81(1-2): 17–29.
- Smallbone, N. 2021. Twee: an equational theorem prover. In *International Conference on Automated Deduction*, 602–613. Springer.
- Tao, T. 2026. Mathematics Distillation Challenge – Equational Theories. <https://terrytao.wordpress.com/2026/03/13/mathematics-distillation-challenge-equational-theories/>.
- Tsoukalas, G.; Lee, J.; Jennings, J.; Xin, J.; Ding, M.; Jennings, M.; Thakur, A.; and Chaudhuri, S. 2024. Putnam-bench: Evaluating neural theorem-provers on the putnam mathematical competition. *Advances in Neural Information Processing Systems*, 37: 11545–11569.
- White, C.; Dooley, S.; Roberts, M.; Pal, A.; Feuer, B.; Jain, S.; Schwartz-Ziv, R.; Jain, N.; Saifullah, K.; Dey, S.; et al. 2025. Livebench: A challenging, contamination-limited llm benchmark. In *International Conference on Learning Representations*, volume 2025, 91595–91631.
- Xin, H.; Guo, D.; Shao, Z.; Ren, Z.; Zhu, Q.; Liu, B.; Ruan, C.; Li, W.; and Liang, X. 2024. Deepseek-prover: Advancing theorem proving in llms through large-scale synthetic data. *arXiv preprint arXiv:2405.14333*.
- Xu, C.; Guan, S.; Greene, D.; Kechadi, M.; et al. 2024. Benchmark data contamination of large language models: A survey. *arXiv preprint arXiv:2406.04244*.
- Yang, K.; Swope, A.; Gu, A.; Chalamala, R.; Song, P.; Yu, S.; Godil, S.; Prenger, R. J.; and Anandkumar, A. 2023. Leandojo: Theorem proving with retrieval-augmented language models. *Advances in Neural Information Processing Systems*, 36: 21573–21612.
- Zheng, K.; Han, J. M.; and Polu, S. 2021. Minif2f: a cross-system benchmark for formal olympiad-level mathematics. *arXiv preprint arXiv:2109.00110*.
- Zhu, K.; Chen, J.; Wang, J.; Gong, N.; Yang, D.; and Xie, X. 2024. Dyval: Dynamic evaluation of large language models for reasoning tasks. In *International Conference on Learning Representations*, volume 2024, 18091–18128.